

```
# Means-End Analysis (MEA)

class MeansEndAnalysis:
    def __init__(self, operators):
        self.operators = operators

    def solve(self, current, goal):
        print(f"Current State: {current} | Goal State: {goal}")

        if current == goal:
            return []

        # Find difference
        diff = self.find_difference(current, goal)
        if not diff:
            return []

        # Find operator that can resolve the difference
        op = self.select_operator(diff)
        if not op:
            print(f"No operator found to resolve difference: {diff}")
            return None
```

```
# Check preconditions
preconditions_path = self.solve(current, op['precond'])
if preconditions_path is None:
    return None

# Apply operator
new_state = op['effect']

# Continue solving
remaining_path = self.solve(new_state, goal)
if remaining_path is None:
    return None

return preconditions_path + [op['name']] + remaining_path

def find_difference(self, current, goal):
    for key in goal:
        if current.get(key) != goal[key]:
            return (key, goal[key])
    return None
```

```
def select_operator(self, diff):
    key, val = diff
    for op in self.operators:
        if op['effect'].get(key) == val:
            return op
    return None
```

Example Usage

```
if __name__ == "__main__":
```

```
    operators = [
        {
            'name': 'Drive_Car',
            'precond': {
                'has_car': True,
                'at_home': True
            },
            'effect': {
                'at_work': True,
                'at_home': False
            }
        },
    ]
```

```
    }  
  },  
  {  
    'name': 'Buy_Car',  
    'precond': {  
      'has_money': True,  
      'has_car': False  
    },  
    'effect': {  
      'has_car': True  
    }  
  }  
]  
  
current_state = {
```

```
  'has_money': True,  
  'has_car': False,  
  'at_home': True,  
  'at_work': False  
}
```

```
goal state = {
```

```
goal_state = {  
    'at_work': True  
}
```

```
mea = MeansEndAnalysis(operators)  
plan = mea.solve(current_state, goal_state)
```

```
print("\nExecution Plan:", plan)  
print("Moulya M P")  
print("24BECS167")
```

[Share](#)[Run](#)

Output

[Clear](#)

```
Current State: {'has_money': True, 'has_car': False, 'at_home': True, 'at_work':  
False} | Goal State: {'at_work': True}  
Current State: {'has_money': True, 'has_car': False, 'at_home': True, 'at_work':  
False} | Goal State: {'has_car': True, 'at_home': True}  
Current State: {'has_money': True, 'has_car': False, 'at_home': True, 'at_work':  
False} | Goal State: {'has_money': True, 'has_car': False}  
Current State: {'has_car': True} | Goal State: {'has_car': True, 'at_home': True}  
No operator found to resolve difference: ('at_home', True)
```

Execution Plan: None

Moulya M P

24BECS167

=== Code Execution Successful ===